

KPI-RAG: Complete Implementation Guide

AI Network Incident Predictor & Root-Cause Assistant

From Zero to Working System — Step by Step

How to use this guide:

Follow every step in order. Do not skip steps.

Every step tells you WHAT to do, HOW to do it, and WHY.

Paste the output of every step here in Claude before moving to the next.

PROJECT STRUCTURE

Create this folder structure on your machine:

```
kpi_rag/  
  data/           raw dataset files  
  notebooks/      exploration and experiments  
    01_load_data.ipynb  
    02_explore_data.ipynb  
    03_feature_engineering.ipynb  
    04_model_training.ipynb  
    05_shap_analysis.ipynb  
    06_rag_pipeline.ipynb  
    07_evaluation.ipynb  
  src/           production code  
    dataset_builder.py  
    feature_engineering.py  
    models.py  
    shap_explainer.py  
    rag_pipeline.py  
    llm_explainer.py
```

```
evaluator.py
app/          Streamlit dashboard
  app.py
artifacts/    saved models and tables
  models/
  alignment_table/
  chromadb/
requirements.txt
README.md
```

PHASE 0 — ENVIRONMENT SETUP

Goal: Have a working Python environment with all libraries installed.

Step 0.1 — Install VS Code

Download from: <https://code.visualstudio.com>

Install the following extensions:

- Python (by Microsoft)
- Jupyter (by Microsoft)
- GitLens (optional but useful)

Why VS Code: It runs Jupyter notebooks, Python scripts, and terminal in one place. You can also connect to the university HPC cluster from it later.

Step 0.2 — Create the project folder

Open terminal in VS Code (Ctrl + `) and run:

```
mkdir kpi_rag
cd kpi_rag
python -m venv venv
```

Why virtual environment: Keeps your project libraries separate from other Python projects on your machine. Avoids version conflicts.

Step 0.3 — Activate the virtual environment

On Windows:

```
venv\Scripts\activate
```

On Mac/Linux:

```
source venv/bin/activate
```

You should see `(venv)` at the start of your terminal line. This means the environment is active.

Step 0.4 — Install all required libraries

```
pip install datasets pandas numpy scikit-learn shap \
    chromadb sentence-transformers \
    streamlit plotly imbalanced-learn \
    jupyter ipykernel openai python-dotenv
```

What each library does:

- `datasets` — loads TelecomTS from HuggingFace
 - `pandas numpy` — data manipulation
 - `scikit-learn` — Random Forest models
 - `shap` — explainability (SHAP TreeExplainer)
 - `chromadb` — vector database for RAG
 - `sentence-transformers` — text embeddings
 - `streamlit plotly` — dashboard
 - `imbalanced-learn` — handles class imbalance
 - `jupyter ipykernel` — run notebooks in VS Code
 - `openai` — API calls to GPT or Groq
 - `python-dotenv` — manage API keys safely
-

Step 0.5 — Create requirements.txt

```
pip freeze > requirements.txt
```

Why: Lets your teammates install the exact same versions on their machines.

Step 0.6 — Create the folder structure

```
mkdir data notebooks src app artifacts
mkdir artifacts/models artifacts/alignment_table artifacts/chromadb
```

PHASE 1 — DATA LOADING AND VALIDATION

Goal: Load TelecomTS, confirm it is correct, understand its structure.

Timeline: Week 1, Day 1-2

Step 1.1 — Create notebook 01_load_data.ipynb

Open VS Code, create a new file: notebooks/01_load_data.ipynb

Cell 1 — Load the dataset:

```
from datasets import load_dataset
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

print("Loading TelecomTS from HuggingFace...")
print("This may take 5-10 minutes on first load...")

dataset = load_dataset(
    "AliMaatouk/TelecomTS",
    data_files={"full": "**/chunked.jsonl"}
)

df = dataset["full"].to_pandas()

print(f"\n Dataset loaded successfully")
```

```
print(f"Total rows: {len(df):,}")
print(f"Total columns: {len(df.columns)}")
print(f"Columns: {df.columns.tolist()}")
```

Expected output:

```
Total rows: 32,000
Total columns: 9
Columns: ['start_time', 'end_time', 'sampling_rate',
          'KPIs', 'description', 'anomalies',
          'statistics', 'labels', 'QnA']
```

Why: Confirms the dataset loaded correctly and has the right shape.

Cell 2 — Check time range:

```
df['start_time'] = pd.to_datetime(df['start_time'])
df['end_time'] = pd.to_datetime(df['end_time'])

print(f"Date range: {df['start_time'].min()} to {df['start_time'].max()}")
print(f"Duration: {df['start_time'].max() - df['start_time'].min()}")
print(f"Sample rate: {df['sampling_rate'].mode()[0]} ms")
```

Why: Confirms the data covers a meaningful time period and sampling rate is 10ms as expected.

Cell 3 — Check anomaly distribution:

```
import re

def get_anomaly_type(text):
    match = re.search(r"type':\s*('[^']+)'", str(text))
    return match.group(1) if match else 'Normal'

def get_anomaly_status(text):
    return 'Anomaly' if "'exists': True" in str(text) else 'Normal'

df['anomaly_type'] = df['anomalies'].apply(get_anomaly_type)
df['anomaly_status'] = df['anomalies'].apply(get_anomaly_status)
```

```
print("=== ANOMALY STATUS ===")
print(df['anomaly_status'].value_counts())
print(f"\nPercentage anomalous: {(df['anomaly_status']=='Anomaly').mean()*100:.1f}%")

print("\n=== ANOMALY TYPES ===")
print(df['anomaly_type'].value_counts())
print(f"\nTotal unique types: {df['anomaly_type'].nunique()}")
```

Expected output:

```
Normal    30,765
Anomaly    1,235
Percentage anomalous: 3.9%

Anomaly types: Jamming (279), Co-Channel Interference...
Total unique types: 12 (11 anomaly types + Normal)
```

Why: Confirms the 96/4 class imbalance and 11 anomaly types. This drives your model design choices.

Cell 4 — Save the dataframe:

```
# Save to parquet for fast reloading
df.to_parquet('data/telecomts_full.parquet', index=False)
print("  Saved to data/telecomts_full.parquet")
print(f"File size: {pd.io.common.get_handle('data/telecomts_full.parquet', 'rb').raw.read(
```

Why: Parquet loads 10x faster than reloading from HuggingFace every time.

Step 1.2 — Create notebook 02_explore_data.ipynb

Cell 1 — Load saved data:

```
import pandas as pd
import numpy as np
import re
```

```
df = pd.read_parquet('data/telecomts_full.parquet')
print(f"Loaded {len(df):,} rows")
```

Cell 2 — Inspect one anomalous sample in full:

```
# Get first anomalous sample
anomalous = df[df['anomaly_status'] == 'Anomaly'].iloc[0]

print("=== ANOMALY SAMPLE ===")
print(f"Start: {anomalous['start_time']}")
print(f>Type: {anomalous['anomaly_type']}")
print(f"\n--- KPIs (first 3 values per channel) ---")

import ast
kpis = anomalous['KPIs']
print(str(kpis)[:500])

print(f"\n--- Anomaly Info ---")
print(str(anomalous['anomalies'])[:500])

print(f"\n--- Description ---")
print(str(anomalous['description'][:300]))

print(f"\n--- QnA (first entry) ---")
print(str(anomalous['QnA'][:300]))
```

Why: Understanding what one full sample looks like is essential before writing any feature engineering code.

Cell 3 — Extract KPI channel names:

```
def get_kpi_names(kpi_text):
    """Extract KPI channel names from one sample"""
    matches = re.findall(r"(\w+):s*array", str(kpi_text))
    return matches

sample_kpi_names = get_kpi_names(df['KPIs'].iloc[0])
print(f"KPI channels ({len(sample_kpi_names)}):")
```

```
for name in sample_kpi_names:
    print(f" - {name}")
```

Expected output:

```
KPI channels (16):
- RSRP
- DL_BLER
- UL_BLER
- UL_MCS
...
```

Why: These are your 16 numerical feature channels. You need their exact names for the 582-dim feature vector.

Cell 4 — Extract one troubleshooting ticket:

```
# Get a troubleshooting ticket from an anomalous sample
anomalous_samples = df[df['anomaly_status'] == 'Anomaly']

ticket_sample = anomalous_samples.iloc[0]
anomaly_str = str(ticket_sample['anomalies'])

# Find troubleshooting ticket content
ticket_match = re.search(r"troubleshooting_tickets':\s*({.*})", anomaly_str, re.DOTALL)
if ticket_match:
    print("=== TROUBLESHOOTING TICKET ===")
    print(ticket_match.group(1)[:600])
else:
    print("Ticket format:")
    print(anomaly_str[:800])
```

Why: These 1,235 tickets are your RAG corpus. You need to confirm they exist and understand their structure before building ChromaDB.

Cell 5 — Check labels column:

```
print("=== LABELS COLUMN ===")
print(df['labels'].iloc[0])
```



```
print(f"\nUnique label structures: {df['labels'].nunique()} types")
```

Why: Labels contain zone, application type, and mobility info — useful as additional features.

Cell 6 — Missing values check:

```
print("=== MISSING VALUES ===")
missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(2)
result = pd.DataFrame({'Count': missing, 'Percentage': missing_pct})
print(result[result['Count'] > 0] if (missing > 0).any() else "No missing values found  ")
```

PHASE 2 — FEATURE ENGINEERING

Goal: Convert raw 128-timestep KPI arrays into 582-dimensional feature vectors.

Timeline: Week 2-3

Step 2.1 — Create src/feature_engineering.py

```
"""
Feature Engineering for KPI-RAG
Converts raw TelecomTS samples into 582-dimensional feature vectors.

Feature vector composition:
- 64 dims: Pre-computed statistics (mean, std, min, max × 16 channels)
- 256 dims: Patchwise scale statistics (8 patches × mean+std × 16 channels)
- 256 dims: First-order differences (8 patches × mean+std × 16 channels)
- 6 dims: Categorical encoding (UL/DL Protocol one-hot)
Total: 582 dimensions
"""

import numpy as np
import pandas as pd
```

```

import re
from typing import List, Tuple

# These are the 16 numerical KPI channels in TelecomTS
KPI_CHANNELS = [
    'RSRP', 'DL_BLER', 'DL_MCS', 'UL_BLER', 'UL_MCS',
    'UL_NPRB', 'UL_SNR', 'TX_Bytes', 'RX_Bytes',
    'Estimated_UL_Buffer', 'PRBs_DL_Current', 'PRBs_UL_Current',
    'PRB_Utilization_DL', 'PRB_Utilization_UL',
    'UL_NumberOfPackets', 'DL_NumberOfPackets'
]

PROTOCOL_CHANNELS = ['UL_Protocol', 'DL_Protocol']
PROTOCOL_VALUES = ['TCP', 'UDP', 'None'] # 3 categories × 2 protocols = 6 dims

N_PATCHES = 8 # Split 128 timesteps into 8 patches
WINDOW_SIZE = 128 # Timesteps per window

def extract_kpi_array(kpi_text: str, channel_name: str) -> np.ndarray:
    """
    Extract one KPI channel as a numpy array from raw text.

    Why: KPIs are stored as nested Python objects in the JSONL.
    We use regex to extract the array values safely.
    """
    pattern = rf"{channel_name}:\s*array\(\[([.*?])\]"
    match = re.search(pattern, str(kpi_text), re.DOTALL)
    if not match:
        return np.zeros(WINDOW_SIZE)

    values_str = match.group(1)
    values = re.findall(r'[-+]?[d*\.]?[d+](?:[eE]([-+]?[d+])?)', values_str)
    arr = np.array([float(v) for v in values[:WINDOW_SIZE]])

    # Pad with zeros if shorter than expected
    if len(arr) < WINDOW_SIZE:
        arr = np.pad(arr, (0, WINDOW_SIZE - len(arr)))

    return arr

```

```
def compute_precomputed_stats(kpi_text: str) -> np.ndarray:
```

```
    """
```

Component 1: Pre-computed statistics — 64 dimensions.

For each of 16 channels: mean, std, min, max

16 channels × 4 statistics = 64 dimensions

Why: Zero computation cost — captures global channel behavior.

NOTE: We re-compute from raw values to avoid using the

TelecomTS 'statistics' field (which may contain label-derived info).

```
    """
```

```
    features = []
```

```
    for channel in KPI_CHANNELS:
```

```
        arr = extract_kpi_array(kpi_text, channel)
```

```
        features.extend([
```

```
            np.mean(arr),
```

```
            np.std(arr),
```

```
            np.min(arr),
```

```
            np.max(arr)
```

```
        ])
```

```
    return np.array(features) # shape: (64,)
```

```
def compute_patchwise_scale(kpi_text: str) -> np.ndarray:
```

```
    """
```

Component 2: Patchwise scale statistics — 256 dimensions.

Split 128 timesteps into 8 patches of 16 timesteps each.

For each patch per channel: mean, std

8 patches × 2 statistics × 16 channels = 256 dimensions

Why: Preserves absolute magnitude within each patch.

Captures WHEN in the window the KPI changed, not just that it changed.

Example: RSRP dropping in patch 5 vs patch 2 is diagnostically different.

IMPORTANT: Do NOT normalize. Scale preservation is the point.

```
    """
```

```
    patch_size = WINDOW_SIZE // N_PATCHES # 128 / 8 = 16 timesteps per patch
```

```
features = []
```

```
for channel in KPI_CHANNELS:
```

```
    arr = extract_kpi_array(kpi_text, channel)
```

```
    patches = arr.reshape(N_PATCHES, patch_size) # shape: (8, 16)
```

```
    for patch in patches:
```

```
        features.append(np.mean(patch))
```

```
        features.append(np.std(patch))
```

```
return np.array(features) # shape: (256,)
```

```
def compute_first_order_differences(kpi_text: str) -> np.ndarray:
```

```
    """
```

Component 3: First-order differences — 256 dimensions.

Compute diff array first, then apply patchwise statistics.

8 patches × 2 statistics × 16 channels = 256 dimensions

Why: Captures rate-of-change, not just level.

`diff([10, 10, 10, 45, 46]) = [0, 0, 35, 1]`

This detects sudden jumps that mean/std miss.

```
    """
```

```
    patch_size = WINDOW_SIZE // N_PATCHES
```

```
    features = []
```

```
    for channel in KPI_CHANNELS:
```

```
        arr = extract_kpi_array(kpi_text, channel)
```

```
        diff_arr = np.diff(arr, prepend=arr[0]) # shape: (128,)
```

```
        patches = diff_arr.reshape(N_PATCHES, patch_size)
```

```
        for patch in patches:
```

```
            features.append(np.mean(patch))
```

```
            features.append(np.std(patch))
```

```
    return np.array(features) # shape: (256,)
```

```
def compute_categorical_encoding(kpi_text: str) -> np.ndarray:
```

```
"""
```

Component 4: Categorical encoding — 6 dimensions.

One-hot encode UL_Protocol and DL_Protocol.

Each has 3 possible values: TCP, UDP, None

2 protocols × 3 values = 6 dimensions

Why: Protocol state under fault is diagnostic.

TCP retransmissions behave differently from UDP drops.

```
"""
```

```
features = np.zeros(6)
```

```
kpi_str = str(kpi_text)
```

```
for i, protocol in enumerate(['UL_Protocol', 'DL_Protocol']):
```

```
    pattern = rf"{protocol}:\s*(\w+)"
```

```
    match = re.search(pattern, kpi_str)
```

```
    if match:
```

```
        value = match.group(1)
```

```
        if value in PROTOCOL_VALUES:
```

```
            idx = PROTOCOL_VALUES.index(value)
```

```
            features[i * 3 + idx] = 1.0
```

```
return features # shape: (6,)
```

```
def build_feature_vector(kpi_text: str) -> np.ndarray:
```

```
    """
```

Build the complete 582-dimensional feature vector for one sample.

Composition:

[0:64] — pre-computed statistics

[64:320] — patchwise scale statistics

[320:576] — first-order differences

[576:582] — categorical encoding

```
    """
```

```
stats = compute_precomputed_stats(kpi_text) # 64 dims
```

```
scale = compute_patchwise_scale(kpi_text) # 256 dims
```

```
diffs = compute_first_order_differences(kpi_text) # 256 dims
```

```
cats = compute_categorical_encoding(kpi_text) # 6 dims
```

```
vector = np.concatenate([stats, scale, diffs, cats])
assert vector.shape == (582,), f"Expected (582,), got {vector.shape}"

return vector
```

```
def build_feature_matrix(df: pd.DataFrame,
                        verbose: bool = True) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
    """
    Build feature matrix for all samples.

    Returns:
        X: feature matrix, shape (n_samples, 582)
        y_binary: binary labels (0=Normal, 1=Anomaly)
        y_multiclass: 12-class labels (0=Normal, 1-11=fault types)
    """
    n = len(df)
    X = np.zeros((n, 582))

    # Build anomaly type to integer mapping
    anomaly_types = df['anomaly_type'].unique().tolist()
    if 'Normal' in anomaly_types:
        anomaly_types.remove('Normal')
    anomaly_types = sorted(anomaly_types)

    type_to_int = {'Normal': 0}
    type_to_int.update({t: i+1 for i, t in enumerate(anomaly_types)})

    y_binary = np.zeros(n, dtype=int)
    y_multiclass = np.zeros(n, dtype=int)

    for i, (_, row) in enumerate(df.iterrows()):
        if verbose and i % 1000 == 0:
            print(f"Processing sample {i:,}/{n:,}...")

        X[i] = build_feature_vector(row['KPIs'])
        y_binary[i] = 1 if row['anomaly_status'] == 'Anomaly' else 0
        y_multiclass[i] = type_to_int.get(row['anomaly_type'], 0)

    print(f"\n Feature matrix built: {X.shape}")
```

```
print(f"Binary labels: {np.bincount(y_binary)}")
print(f"Multiclass labels: {np.bincount(y_multiclass)}")

return X, y_binary, y_multiclass, type_to_int
```

Step 2.2 — Create notebook 03_feature_engineering.ipynb

Cell 1 — Test feature extraction on one sample:

```
import pandas as pd
import numpy as np
import sys
sys.path.append('../src')
from feature_engineering import build_feature_vector, KPI_CHANNELS

df = pd.read_parquet('../data/telecomts_full.parquet')

# Test on one sample
sample = df.iloc[0]
vector = build_feature_vector(sample['KPIs'])

print(f"Feature vector shape: {vector.shape}")
print(f"Expected: (582,)")
print(f"\nComponent breakdown:")
print(f" Stats [0:64]: min={vector[0:64].min():.3f}, max={vector[0:64].max():.3f}")
print(f" Scale [64:320]: min={vector[64:320].min():.3f}, max={vector[64:320].max():.3f}")
print(f" Diffs [320:576]: min={vector[320:576].min():.3f}, max={vector[320:576].max():.3f}")
print(f" Cats [576:582]: {vector[576:582]}")
```

Cell 2 — Build full feature matrix (takes 10-20 minutes):

```
from feature_engineering import build_feature_matrix
import numpy as np

X, y_binary, y_multiclass, type_to_int = build_feature_matrix(df, verbose=True)

# Save to disk
np.save('../data/X_features.npy', X)
```

```

np.save('../data/y_binary.npy', y_binary)
np.save('../data/y_multiclass.npy', y_multiclass)

import json
with open('../data/type_to_int.json', 'w') as f:
    json.dump(type_to_int, f, indent=2)

print("\n All arrays saved to data/")
print(f"X shape:      {X.shape}")
print(f"y_binary shape: {y_binary.shape}")
print(f"y_multiclass shape: {y_multiclass.shape}")

```

Cell 3 — Create train/test split (leakage-safe by recording):

```

from sklearn.model_selection import train_test_split
import numpy as np

# Load saved arrays
X = np.load('../data/X_features.npy')
y_binary = np.load('../data/y_binary.npy')
y_multiclass = np.load('../data/y_multiclass.npy')

# IMPORTANT: Split at recording level not window level
# This prevents data leakage from 75% window overlap (stride=32, window=128)
# Each JSONL file = one recording session
# For now we use stratified split as approximation
# Week 1 task: check if TelecomTS has official split

X_train, X_test, y_bin_train, y_bin_test, y_mc_train, y_mc_test = train_test_split(
    X, y_binary, y_multiclass,
    test_size=0.2,
    random_state=42,
    stratify=y_binary # stratify by binary label to preserve 96/4 ratio
)

print(f"Train size: {X_train.shape[0]:,}")
print(f"Test size: {X_test.shape[0]:,}")
print(f"\nTrain binary distribution: {np.bincount(y_bin_train)}")
print(f"Test binary distribution: {np.bincount(y_bin_test)}")

```



```
# Save splits
np.save('../data/X_train.npy', X_train)
np.save('../data/X_test.npy', X_test)
np.save('../data/y_bin_train.npy', y_bin_train)
np.save('../data/y_bin_test.npy', y_bin_test)
np.save('../data/y_mc_train.npy', y_mc_train)
np.save('../data/y_mc_test.npy', y_mc_test)

print("\n Train/test splits saved")
```

PHASE 3 — MODEL TRAINING

Goal: Train Model 1 (binary detector) and Model 2 (11-class classifier).

Timeline: Week 4-5

Step 3.1 — Create notebook 04_model_training.ipynb

Cell 1 — Load data and set up:

```
import numpy as np
import json
from sklearn.ensemble import RandomForestClassifier, IsolationForest
from sklearn.metrics import (classification_report, confusion_matrix,
                             f1_score, precision_score, recall_score,
                             roc_auc_score)
from sklearn.model_selection import RandomizedSearchCV
import warnings
warnings.filterwarnings('ignore')

X_train = np.load('../data/X_train.npy')
X_test = np.load('../data/X_test.npy')
y_bin_train = np.load('../data/y_bin_train.npy')
y_bin_test = np.load('../data/y_bin_test.npy')
y_mc_train = np.load('../data/y_mc_train.npy')
```

```

y_mc_test = np.load('../data/y_mc_test.npy')

with open('../data/type_to_int.json') as f:
    type_to_int = json.load(f)

int_to_type = {v: k for k, v in type_to_int.items()}

print(f"Train: {X_train.shape}, Test: {X_test.shape}")
print(f"Anomaly types: {type_to_int}")

```

Cell 2 — Model 1: Binary Anomaly Detector:

```

# Rebalance training data to 80/20 normal/anomalous
# (consistent with TelecomTS paper evaluation protocol)
from imblearn.under_sampling import RandomUnderSampler

rus = RandomUnderSampler(sampling_strategy=0.25, random_state=42)
# 0.25 means minority/majority = 0.25 20% anomalous / 80% normal
X_train_bal, y_bin_train_bal = rus.fit_resample(X_train, y_bin_train)

print(f"Rebalanced training set: {np.bincount(y_bin_train_bal)}")
print(f"Ratio: {np.bincount(y_bin_train_bal)[1]/len(y_bin_train_bal)*100:.1f}% anomalous")

# Train Random Forest binary detector
print("\nTraining Model 1 (Binary Detector)...")
model1 = RandomForestClassifier(
    n_estimators=200,
    class_weight='balanced', # additional compensation within rebalanced set
    random_state=42,
    n_jobs=-1 # use all CPU cores
)
model1.fit(X_train_bal, y_bin_train_bal)

# Evaluate on NATURAL distribution (not rebalanced)
y_pred_bin = model1.predict(X_test)
y_prob_bin = model1.predict_proba(X_test)[:, 1]

print("\n=== MODEL 1 RESULTS ===")
print(f"Precision: {precision_score(y_bin_test, y_pred_bin):.3f}")

```

```
print(f"Recall:  {recall_score(y_bin_test, y_pred_bin):.3f}")
print(f"F1:      {f1_score(y_bin_test, y_pred_bin):.3f}")
print(f"AUC-ROC: {roc_auc_score(y_bin_test, y_prob_bin):.3f}")
print(f"\nReference: Mantis F1=0.800 (TelecomTS Table 2, 80/20 balanced eval)")
print(f"\n{classification_report(y_bin_test, y_pred_bin, target_names=['Normal', 'Anomaly'])}
```

Cell 3 — Model 1: Isolation Forest baseline:

```
# Train Isolation Forest on normal samples only (unsupervised baseline)
X_normal = X_train[y_bin_train == 0]

print("Training Isolation Forest baseline...")
iso_forest = IsolationForest(
    n_estimators=100,
    contamination=0.04, # expected anomaly rate
    random_state=42,
    n_jobs=-1
)
iso_forest.fit(X_normal)

# Predict: -1 = anomaly, 1 = normal  convert to 0/1
y_pred_iso = (iso_forest.predict(X_test) == -1).astype(int)

print("\n=== ISOLATION FOREST RESULTS ===")
print(f"Precision: {precision_score(y_bin_test, y_pred_iso):.3f}")
print(f"Recall:    {recall_score(y_bin_test, y_pred_iso):.3f}")
print(f"F1:       {f1_score(y_bin_test, y_pred_iso):.3f}")
```

Cell 4 — Model 2: 11-class fault classifier:

```
# Model 2 runs only on anomalous samples
# Exclude Jamming from training (held-out generalization probe)
JAMMING_CLASS = type_to_int.get('Jamming', -1)

# Get anomalous training samples (excluding Jamming)
anomaly_mask_train = (y_mc_train > 0) & (y_mc_train != JAMMING_CLASS)
X_mc_train = X_train[anomaly_mask_train]
y_mc_train_filtered = y_mc_train[anomaly_mask_train]
```

```

print(f"Model 2 training samples: {len(X_mc_train):,} (Jamming excluded)")
print(f"Class distribution: {np.bincount(y_mc_train_filtered)}")

# Get anomalous test samples (excluding Jamming — separate evaluation)
anomaly_mask_test = (y_mc_test > 0) & (y_mc_test != JAMMING_CLASS)
X_mc_test = X_test[anomaly_mask_test]
y_mc_test_filtered = y_mc_test[anomaly_mask_test]

# Train Model 2
print("\nTraining Model 2 (11-class Fault Classifier)...")
model2 = RandomForestClassifier(
    n_estimators=200,
    class_weight='balanced',
    random_state=42,
    n_jobs=-1
)
model2.fit(X_mc_train, y_mc_train_filtered)

y_pred_mc = model2.predict(X_mc_test)

print("\n=== MODEL 2 RESULTS ===")
print(f"F1-macro: {f1_score(y_mc_test_filtered, y_pred_mc, average='macro'):.3f}")
print(f"Accuracy: {(y_pred_mc == y_mc_test_filtered).mean():.3f}")
print(f"\nReference: Toto accuracy=0.848 (TelecomTS Table 4)")
print(f"\n{classification_report(y_mc_test_filtered, y_pred_mc, target_names=[int_to_type

```

Cell 5 — Jamming generalization probe:

```

# Evaluate on Jamming samples (zero training exposure)
jamming_mask_test = y_mc_test == JAMMING_CLASS
X_jamming = X_test[jamming_mask_test]
y_jamming = y_mc_test[jamming_mask_test]

if len(X_jamming) > 0:
    y_pred_jamming = model2.predict(X_jamming)
    jamming_accuracy = (y_pred_jamming == JAMMING_CLASS).mean()
    print(f"\n=== JAMMING GENERALIZATION PROBE ===")
    print(f"Jamming test samples: {len(X_jamming)}")

```

```
print(f"Correctly classified as Jamming: {jamming_accuracy*100:.1f}%")
print("(This tests zero-shot generalization to real over-the-air anomaly)")
```

Cell 6 — Save models:

```
import joblib

joblib.dump(model1, '../artifacts/models/model1_binary.pkl')
joblib.dump(model2, '../artifacts/models/model2_multiclass.pkl')
joblib.dump(iso_forest, '../artifacts/models/isolation_forest.pkl')

print("  Models saved to artifacts/models/")
```

PHASE 4 — SHAP EXPLAINABILITY

Goal: Extract SHAP values and build SHAP-to-query bridge.

Timeline: Week 5-6

Step 4.1 — Create notebook 05_shap_analysis.ipynb

Cell 1 — Compute SHAP values:

```
import shap
import numpy as np
import joblib
import json

model1 = joblib.load('../artifacts/models/model1_binary.pkl')
model2 = joblib.load('../artifacts/models/model2_multiclass.pkl')
X_test = np.load('../data/X_test.npy')

# Feature names for SHAP
from feature_engineering import KPI_CHANNELS

feature_names = []
```

```

# Component 1: stats
for ch in KPI_CHANNELS:
    for stat in ['mean', 'std', 'min', 'max']:
        feature_names.append(f"{ch}_{stat}")
# Component 2: patchwise scale
for ch in KPI_CHANNELS:
    for p in range(8):
        feature_names.append(f"{ch}_patch{p}_mean")
        feature_names.append(f"{ch}_patch{p}_std")
# Component 3: first-order differences
for ch in KPI_CHANNELS:
    for p in range(8):
        feature_names.append(f"{ch}_diff_patch{p}_mean")
        feature_names.append(f"{ch}_diff_patch{p}_std")
# Component 4: categorical
feature_names.extend(['UL_TCP', 'UL_UDP', 'UL_None',
                     'DL_TCP', 'DL_UDP', 'DL_None'])

print(f"Total feature names: {len(feature_names)}")

# Compute SHAP values for Model 1
print("\nComputing SHAP values for Model 1...")
explainer1 = shap.TreeExplainer(model1)
shap_values1 = explainer1.shap_values(X_test[:100]) # start with 100 samples

print(" SHAP values computed")
print(f"SHAP values shape: {np.array(shap_values1).shape}")

```

Cell 2 — SHAP-to-query bridge:

```

def shap_to_query(shap_values: np.ndarray,
                  predicted_fault_type: str,
                  feature_names: list,
                  top_k: int = 3) -> str:
    """
    Convert SHAP values to a natural language retrieval query.

    This is Novel Contribution 4 — the bridge between ML and RAG.

```

Why: Instead of querying ChromaDB with raw fault type name, we construct a semantic description based on which KPIs drove the prediction and in which direction.

Example output:

"Co-Channel Interference anomaly detected.

Primary affected KPIs: DL_BLER (increased),
UL_SNR (decreased), PRB_Utilization_DL (increased).

Signal direction: uplink quality degraded with elevated error rates."

"""

Get absolute SHAP values for this prediction

abs_shap = np.abs(shap_values)

Get top-k feature indices

top_k_idx = np.argsort(abs_shap)[-top_k::-1]

Map feature indices back to KPI channel names

top_kpis = []

for idx in top_k_idx:

feature = feature_names[idx]

channel = feature.split('_')[0] # extract channel name

direction = 'increased' if shap_values[idx] > 0 else 'decreased'

top_kpis.append(f"{channel} ({direction})")

Build query string

query = (

f"{predicted_fault_type} anomaly detected. "

f"Primary affected KPIs: {' '.join(top_kpis)}. "

f"Retrieve: root cause, 3GPP standard reference, and recommended resolution."

)

return query, top_kpis

Test the function

sample_shap = shap_values1[0] if isinstance(shap_values1, list) else shap_values1[0]

query, top_kpis = shap_to_query(

sample_shap,

"Jamming",

feature_names,

```
top_k=3
)

print("=== SHAP-TO-QUERY OUTPUT ===")
print(f"Query: {query}")
print(f"Top KPIs: {top_kpis}")
```

PHASE 5 — RAG PIPELINE

Goal: Build ChromaDB index from TelecomTS tickets and enable retrieval.

Timeline: Week 6-8

Step 5.1 — Create notebook 06_rag_pipeline.ipynb

Cell 1 — Extract all troubleshooting tickets:

```
import pandas as pd
import re
import json

df = pd.read_parquet('../data/telecomts_full.parquet')
anomalous = df[df['anomaly_status'] == 'Anomaly'].copy()

print(f"Anomalous samples: {len(anomalous);}")
print(f"Expected tickets: 1,235")

def extract_ticket(anomaly_text: str) -> dict:
    """Extract troubleshooting ticket from anomaly field"""
    text = str(anomaly_text)

    ticket = {
        'anomaly_type': "",
        'issue': "",
        'symptoms': "",
        'root_cause': "",
```



```

        'resolution': ''
    }

    # Extract anomaly type
    type_match = re.search(r'"type":\s*('[^"]+)"', text)
    if type_match:
        ticket['anomaly_type'] = type_match.group(1)

    # Extract ticket fields
    for field in ['issue', 'symptoms', 'root_cause', 'resolution']:
        pattern = rf'"{field}":\s*('[^"]+)"'
        match = re.search(pattern, text, re.IGNORECASE)
        if match:
            ticket[field] = match.group(1)

    return ticket

# Extract all tickets
tickets = []
for _, row in anomalous.iterrows():
    ticket = extract_ticket(row['anomalies'])
    ticket['sample_id'] = row.name
    tickets.append(ticket)

tickets_df = pd.DataFrame(tickets)
print(f"\nExtracted tickets: {len(tickets_df):,}")
print(f"Unique types: {tickets_df['anomaly_type'].nunique()}")
print(tickets_df['anomaly_type'].value_counts())

tickets_df.to_csv('../data/tickets.csv', index=False)
print("\n Tickets saved to data/tickets.csv")

```

Cell 2 — Build ChromaDB index:

```

import chromadb
from sentence_transformers import SentenceTransformer
import pandas as pd

tickets_df = pd.read_csv('../data/tickets.csv')

```

```

# Initialize ChromaDB
print("Initializing ChromaDB...")
chroma_client = chromadb.PersistentClient(path="../artifacts/chromadb")
collection = chroma_client.get_or_create_collection(
    name="telecomts_tickets",
    metadata={"hnsw:space": "cosine"}
)

# Initialize sentence transformer
print("Loading embedding model...")
encoder = SentenceTransformer('all-MiniLM-L6-v2')

# Build document strings for embedding
documents = []
metadatas = []
ids = []

for i, row in tickets_df.iterrows():
    doc = (
        f"Fault type: {row['anomaly_type']}. "
        f"Issue: {row['issue']}. "
        f"Symptoms: {row['symptoms']}. "
        f"Root cause: {row['root_cause']}. "
        f"Resolution: {row['resolution']}."
    )
    documents.append(doc)
    metadatas.append({
        'anomaly_type': row['anomaly_type'],
        'sample_id': str(row['sample_id'])
    })
    ids.append(f"ticket_{i}")

# Embed and store in batches
print(f"Embedding {len(documents):,} tickets...")
batch_size = 100

for i in range(0, len(documents), batch_size):
    batch_docs = documents[i:i+batch_size]
    batch_meta = metadatas[i:i+batch_size]

```

```

batch_ids = ids[i:i+batch_size]

embeddings = encoder.encode(batch_docs).tolist()

collection.add(
    documents=batch_docs,
    embeddings=embeddings,
    metadatas=batch_meta,
    ids=batch_ids
)

if i % 500 == 0:
    print(f" Processed {i:,}/{len(documents):,}")

print(f"\n ChromaDB indexed {collection.count():,} tickets")

```

Cell 3 — Test retrieval:

```

# Test query
test_query = ("Jamming anomaly detected. "
              "Primary affected KPIs: DL_BLER (increased), "
              "UL_SNR (decreased), DL_MCS (decreased). "
              "Retrieve: root cause and 3GPP standard reference.")

query_embedding = encoder.encode([test_query]).tolist()

results = collection.query(
    query_embeddings=query_embedding,
    n_results=3,
    include=['documents', 'metadatas', 'distances']
)

print("=== TOP 3 RETRIEVED TICKETS ===")
for i, (doc, meta, dist) in enumerate(zip(
    results['documents'][0],
    results['metadatas'][0],
    results['distances'][0]
)):
    print(f"\n--- Result {i+1} (similarity: {1-dist:.3f}) ---")

```

```
print(f'Type: {meta['anomaly_type']}')
print(f'Content: {doc[:200]}...')
```

PHASE 6 — LLM INTEGRATION

Goal: Connect SHAP output to ChromaDB retrieval to LLM explanation.

Timeline: Week 8-10

Step 6.1 — Set up API key safely

Create a file called `.env` in your project root:

```
GROQ_API_KEY=your_groq_api_key_here
OPENAI_API_KEY=your_openai_key_here_if_using
```

Add `.env` to `.gitignore` immediately:

```
echo ".env" >> .gitignore
```

Why: Never put API keys in code. They get committed to Git accidentally.

Step 6.2 — Create `src/llm_explainer.py`

```
"""
LLM Explanation Generator for KPI-RAG.
Takes anomaly context + retrieved tickets + alignment table row
and generates a structured JSON root cause explanation.
"""

import os
import json
import re
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
# Valid 3GPP TS series range (series 21-38, three-digit sub-number)
```

```
TS_REGEX = re.compile(r'\bTS\s*(2[1-9]|3[0-8])\.\d{3}\b')
```

```
SYSTEM_PROMPT = """You are a senior 5G network engineer performing root cause ana
```

Given:

1. A detected fault type and the KPI channels that caused it (from SHAP analysis)
2. Similar historical incidents retrieved from the knowledge base
3. The relevant 3GPP/O-RAN standards reference for this fault type

Generate a structured JSON explanation with exactly these five fields:

```
{
  "root_cause": "One sentence describing the root cause",
  "3gpp_reference": "The TS number in format TS XX.XXX Section X.X",
  "oran_component": "The O-RAN component if applicable, else null",
  "recommended_action": "One to three specific diagnostic steps",
  "reasoning_trace": "Two to three sentences explaining your reasoning"
}
```

Rules:

1. Every claim must be grounded in the retrieved tickets or the provided 3GPP reference
2. Do not invent TS numbers — use only the one provided in the standards reference
3. If no 3GPP reference is provided, set 3gpp_reference to null
4. Be specific about KPI values and fault mechanisms
5. Return ONLY valid JSON — no markdown, no explanation outside the JSON"""

```
def build_prompt(fault_type: str,
                 top_kpis: list,
                 retrieved_tickets: list,
                 alignment_row: dict) -> str:
```

```
    """Build the LLM prompt from all available context."""
```

```
    prompt_parts = [
        f"DETECTED FAULT: {fault_type}",
        f"PRIMARY AFFECTED KPIs: {' '.join(top_kpis)}",
        "",
```

```

    "RETRIEVED HISTORICAL INCIDENTS:",
]

for i, ticket in enumerate(retrieved_tickets, 1):
    prompt_parts.append(f" [{i}] {ticket}")

if alignment_row:
    prompt_parts.extend([
        "",
        "STANDARDS REFERENCE (from Fault-to-Standard Alignment Table):",
        f" 3GPP: {alignment_row.get('3gpp_clause', 'N/A')}",
        f" O-RAN: {alignment_row.get('oran_component', 'N/A')}",
        f" Description: {alignment_row.get('description', 'N/A')}"
    ])

prompt_parts.append("\nGenerate the JSON explanation:")

return "\n".join(prompt_parts)

def validate_output(response_text: str) -> dict:
    """
    Validate LLM JSON output.

    Checks:
    1. Valid JSON format
    2. All required fields present
    3. 3GPP TS reference format valid (if present)
    4. TS number exists in alignment table (if present)
    """
    required_fields = ['root_cause', '3gpp_reference',
                       'oran_component', 'recommended_action',
                       'reasoning_trace']

    # Parse JSON
    try:
        output = json.loads(response_text)
    except json.JSONDecodeError:
        # Try to extract JSON from response
        json_match = re.search(r'\{.*\}', response_text, re.DOTALL)

```

```

if json_match:
    output = json.loads(json_match.group())
else:
    raise ValueError("No valid JSON found in response")

# Check required fields
for field in required_fields:
    if field not in output:
        output[field] = None

# Validate TS reference format
ts_ref = output.get('3gpp_reference', '')
if ts_ref and ts_ref != 'null':
    if not TS_REGEX.search(str(ts_ref)):
        output['grounded'] = False
        output['validation_note'] = 'Invalid 3GPP TS format'
    else:
        output['grounded'] = True
else:
    output['grounded'] = False

return output

```

```

def generate_explanation(fault_type: str,
                       top_kpis: list,
                       retrieved_tickets: list,
                       alignment_row: dict,
                       use_groq: bool = True) -> dict:

```

```

"""

```

Generate a grounded root cause explanation.

Tries up to 3 times with decreasing temperature.

Falls back to template if all attempts fail.

```

"""

```

```

prompt = build_prompt(fault_type, top_kpis, retrieved_tickets, alignment_row)

```

```

temperatures = [0.7, 0.3, 0.1]

```

```

for attempt, temp in enumerate(temperatures, 1):

```

```

try:
    if use_groq:
        from openai import OpenAI
        client = OpenAI(
            base_url="https://api.groq.com/openai/v1",
            api_key=os.getenv("GROQ_API_KEY")
        )
        model = "llama-3.1-8b-instant"
    else:
        from openai import OpenAI
        client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
        model = "gpt-4o-mini"

    response = client.chat.completions.create(
        model=model,
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": prompt}
        ],
        temperature=temp,
        response_format={"type": "json_object"}
    )

    output = validate_output(response.choices[0].message.content)
    output['attempt'] = attempt
    output['template_fallback'] = False
    return output

except Exception as e:
    print(f"Attempt {attempt} failed: {e}")
    if attempt == len(temperatures):
        # Template fallback
        return {
            'root_cause': f"Detected {fault_type} based on KPI analysis",
            '3gpp_reference': alignment_row.get('3gpp_clause') if alignment_row else None,
            'oran_component': alignment_row.get('oran_component') if alignment_row else None,
            'recommended_action': alignment_row.get('resolution', 'Contact network operator'),
            'reasoning_trace': f"Template-based explanation. Top KPIs: {' '.join(top_kpis)}",
            'grounded': alignment_row is not None,

```



```
'template_fallback': True
}
```

PHASE 7 — STREAMLIT DASHBOARD

Goal: Build the final user interface.

Timeline: Week 10-11

Step 7.1 — Create app/app.py

```
"""
KPI-RAG Streamlit Dashboard
End-to-end 5G Network Fault Diagnosis Interface
"""

import streamlit as st
import numpy as np
import pandas as pd
import plotly.graph_objects as go
import joblib
import shap
import json
import sys
sys.path.append('../src')

from feature_engineering import build_feature_vector, KPI_CHANNELS

# Page config
st.set_page_config(
    page_title="KPI-RAG: 5G Fault Diagnosis",
    page_icon=" ",
    layout="wide"
)

st.title(" KPI-RAG: 5G Network Fault Diagnosis")
st.markdown("**AI-powered anomaly detection with 3GPP-grounded root cause explan
```

```

# Load models
@st.cache_resource
def load_models():
    model1 = joblib.load('./artifacts/models/model1_binary.pkl')
    model2 = joblib.load('./artifacts/models/model2_multiclass.pkl')
    with open('./data/type_to_int.json') as f:
        type_to_int = json.load(f)
    int_to_type = {v: k for k, v in type_to_int.items()}
    return model1, model2, int_to_type

model1, model2, int_to_type = load_models()

# File upload
uploaded_file = st.file_uploader("Upload a TelecomTS sample (CSV)", type=['csv'])

if uploaded_file:
    # Process and display results
    st.success("File uploaded — running analysis...")

    # Panel 1: Detection result
    col1, col2, col3 = st.columns(3)
    with col1:
        st.metric("Anomaly Detected", "YES", delta="Confidence: 0.94")
    with col2:
        st.metric("Fault Type", "Co-Channel Interference")
    with col3:
        st.metric("Severity", "High")

    # Panel 2: KPI signal plot
    st.subheader("KPI Signal Plot")
    st.info("KPI time-series visualization will appear here")

    # Panel 3: SHAP chart
    st.subheader("Feature Attribution (SHAP)")
    st.info("SHAP waterfall chart will appear here")

    # Panel 4: RCA explanation
    st.subheader("Root Cause Explanation")
    st.info("3GPP-grounded explanation will appear here")

```

```
else:
    st.info("Upload a TelecomTS sample CSV file to begin diagnosis")

# Run with: streamlit run app/app.py
```

PHASE 8 — EVALUATION

Goal: Run all ablations and compute final metrics.

Timeline: Week 12-13

Step 8.1 — Track A: Detection metrics

Already computed in Phase 3. Collect:

- Model 1: Precision, Recall, F1 vs Mantis (0.800)
 - Model 2: Per-class F1-macro, Accuracy vs Toto (0.848)
 - Jamming probe: generalization F1
-

Step 8.2 — Track B: Explanation faithfulness

For 30 stratified random samples:

1. Generate explanation with full system (Condition 3)
 2. Score each on 4 SURE-Score dimensions (1-3 each)
 3. Compute inter-rater agreement
 4. Report mean $\pm$ bootstrap 95% CI
-

Step 8.3 — Track C: Three-condition ablation

Condition	Setup	Metric
Condition 1	No retrieval, no alignment table	Groundedness %
Condition 2	Ticket retrieval only	Groundedness %

Condition 3	Full system (tickets + alignment table)	Groundedness %
-------------	---	----------------

CHECKPOINT SCHEDULE

Week	Checkpoint	What to verify
Week 1	Data loaded	32,000 rows, 9 columns, 11 anomaly types
Week 2	Feature vector	Shape (N, 582), no NaN values
Week 3	Train/test split	Leakage-safe, stratified
Week 5	Model lock	Model 1 F1 > 0.35, Model 2 running
Week 6	Alignment table	10 rows validated against 3GPP docs
Week 8	Layer 1+2 integration	One sample through both models
Week 9	RAG retrieval	precision@3 acceptable
Week 11	Full integration	End-to-end demo running
Week 12	Evaluation	All metrics computed
Week 15	Final submission	Report + code + demo

IMPORTANT RULES

1. **Never normalize KPI values** — scale preservation is the core design decision
 2. **Never use anomalies/labels/QnA/description fields as classifier input** — label leakage
 3. **Model lock at Week 5** — do not keep tuning past this date
 4. **Split by recording session** — not by window (prevents 75% overlap leakage)
 5. **Jamming held out from Model 2 training** — zero-exposure generalization probe
 6. **API keys in .env only** — never in code
 7. **Save everything to disk** — feature matrix, models, ChromaDB, tickets
-

PHASE 3.5 — FAULT-TO-STANDARD ALIGNMENT TABLE

Goal: Build the 10-row table linking each fault type to a specific 3GPP clause and O-RAN component.

Timeline: Week 3–4 (draft), Week 5–6 (validated)

Owner: P3

What is the Alignment Table?

A static JSON file with 10 rows — one per synthetic fault type in TelecomTS.
Each row maps:

- Fault type name specific 3GPP TS clause O-RAN component description

Jamming is excluded — it is a physical RF attack with no corresponding 3GPP clause.

Step 3.5.1 — Create the draft table

Create file: `artifacts/alignment_table/alignment_table_draft.json`

```
[
  {
    "fault_type": "Co-Channel Interference (Severe)",
    "3gpp_clause": "TS 38.211 Section 6.4.1",
    "oran_component": "O-RU",
    "mapping_relation": "governed-by",
    "description": "Severe interference from co-frequency neighboring cells degrading SINR",
    "primary_kpis": ["UL_SNR", "DL_BLER", "UL_BLER"],
    "validated": false,
    "evidence_span": "",
    "validator_1": "",
    "validator_2": "",
    "agreement": ""
  },
  {
    "fault_type": "Co-Channel Interference (Mild)",
    "3gpp_clause": "TS 38.211 Section 6.4.1",
    "oran_component": "O-RU",
    "mapping_relation": "governed-by",
```

```

    "description": "Mild interference from co-frequency neighboring cells",
    "primary_kpis": ["UL_SNR", "DL_BLER"],
    "validated": false,
    "evidence_span": "",
    "validator_1": "",
    "validator_2": "",
    "agreement": ""
  },
  {
    "fault_type": "Antenna Failure",
    "3gpp_clause": "TS 38.104 Section 6.3",
    "oran_component": "O-RU",
    "mapping_relation": "measured-by",
    "description": "Physical antenna hardware failure causing RSRP and SNR degradation",
    "primary_kpis": ["RSRP", "UL_SNR", "DL_BLER", "UL_BLER"],
    "validated": false,
    "evidence_span": "",
    "validator_1": "",
    "validator_2": "",
    "agreement": ""
  },
  {
    "fault_type": "Faulty RF Filters (Temporal)",
    "3gpp_clause": "TS 38.104 Section 6.5",
    "oran_component": "O-RU",
    "mapping_relation": "specified-in",
    "description": "Temporal RF filter degradation causing selective frequency band atten",
    "primary_kpis": ["UL_SNR", "RSRP", "DL_BLER"],
    "validated": false,
    "evidence_span": "",
    "validator_1": "",
    "validator_2": "",
    "agreement": ""
  },
  {
    "fault_type": "Buffer Overflow (Gradual Buildup)",
    "3gpp_clause": "TS 38.321 Section 5.4",
    "oran_component": "O-DU",
    "mapping_relation": "governed-by",
    "description": "Gradual uplink buffer overflow due to sustained resource contention",

```

```

    "primary_kpis": ["Estimated_UL_Buffer", "PRB_Utilization_UL", "UL_BLER"],
    "validated": false,
    "evidence_span": "",
    "validator_1": "",
    "validator_2": "",
    "agreement": ""
  },
  {
    "fault_type": "High Network Congestion (Gradual Buildup)",
    "3gpp_clause": "TS 38.300 Section 16.1",
    "oran_component": "Near-RT RIC",
    "mapping_relation": "governed-by",
    "description": "Gradual traffic overload causing PRB exhaustion and throughput degradation",
    "primary_kpis": ["PRB_Utilization_DL", "PRB_Utilization_UL", "TX_Bytes"],
    "validated": false,
    "evidence_span": "",
    "validator_1": "",
    "validator_2": "",
    "agreement": ""
  },
  {
    "fault_type": "High Network Congestion (Sudden Spike)",
    "3gpp_clause": "TS 38.300 Section 16.1",
    "oran_component": "Near-RT RIC",
    "mapping_relation": "governed-by",
    "description": "Sudden traffic spike causing immediate PRB saturation",
    "primary_kpis": ["PRB_Utilization_DL", "TX_Bytes", "RX_Bytes"],
    "validated": false,
    "evidence_span": "",
    "validator_1": "",
    "validator_2": "",
    "agreement": ""
  },
  {
    "fault_type": "Doppler Shift (Severe)",
    "3gpp_clause": "TS 38.211 Section 6.3",
    "oran_component": "O-RU",
    "mapping_relation": "governed-by",
    "description": "Severe Doppler shift from fast-moving UE causing subcarrier frequency offset",
    "primary_kpis": ["DL_MCS", "UL_MCS", "DL_BLER"],

```

```

    "validated": false,
    "evidence_span": "",
    "validator_1": "",
    "validator_2": "",
    "agreement": ""
  },
  {
    "fault_type": "Faulty Handover Algorithm (Too Frequent)",
    "3gpp_clause": "TS 38.300 Section 9.2",
    "oran_component": "O-CU",
    "mapping_relation": "governed-by",
    "description": "Misconfigured handover thresholds causing excessive handover attempts",
    "primary_kpis": ["RSRP", "UL_SNR", "DL_BLER"],
    "validated": false,
    "evidence_span": "",
    "validator_1": "",
    "validator_2": "",
    "agreement": ""
  },
  {
    "fault_type": "Resource Allocation Bugs",
    "3gpp_clause": "TS 38.214 Section 5.1",
    "oran_component": "O-DU",
    "mapping_relation": "specified-in",
    "description": "Software fault in resource block scheduling causing suboptimal PRB allocation",
    "primary_kpis": ["PRBs_DL_Current", "PRBs_UL_Current", "DL_MCS"],
    "validated": false,
    "evidence_span": "",
    "validator_1": "",
    "validator_2": "",
    "agreement": ""
  }
]

```

Step 3.5.2 — Validation protocol

Two team members independently validate each row:

1. Read the TelecomTS paper description for that fault type
2. Search GSMA telecom-kg-rel19 for the clause: TS XX.XXX

3. Open the actual 3GPP specification document and confirm the clause covers this fault
4. Record the exact quoted sentence as `evidence_span`
5. Set `validated: true` and fill `validator_1` and `validator_2`
6. If disagreement — discuss and resolve against source document
7. If unresolvable — set `agreement: "ambiguous"` and add uncertainty flag

After both validators finish — compute agreement:

```
import json

with open('artifacts/alignment_table/alignment_table_draft.json') as f:
    table = json.load(f)

agreed = sum(1 for row in table if row['validator_1'] == row['validator_2'])
print(f"Agreement: {agreed}/{len(table)} = {agreed/len(table)*100:.0f}%")
```

Step 3.5.3 — Load table at inference time

```
import json

def load_alignment_table(path='artifacts/alignment_table/alignment_table_draft.json'):
    with open(path) as f:
        table = json.load(f)
    # Convert to dict for fast lookup
    return {row['fault_type']: row for row in table}

def get_alignment_row(fault_type: str, table: dict) -> dict:
    """
    Get the 3GPP/O-RAN alignment row for a detected fault type.
    Returns None for Jamming (no applicable standard).
    """
    if fault_type == 'Jamming':
        return None
    return table.get(fault_type, None)
```

PHASE 2.5 — FEATURE ABLATION STUDY

Goal: Quantify the contribution of scale-preserving features (RQ1 and RQ3).

Timeline: Week 12

Owner: P1 + P2

Three configurations to compare:

Config	Features	Dimensions	Purpose
A — Stats only	Pre-computed statistics	64	Baseline — no scale info
B — Stats + Scale	Stats + patchwise scale	320	Scale-aware partial
C — Full (proposed)	All 4 components	582	Our architecture

Step 2.5.1 — Build ablation datasets

Add this to `src/feature_engineering.py` :

```
def build_ablation_datasets(df):
    """Build three feature sets for ablation study."""

    n = len(df)

    # Config A — stats only (64 dims)
    X_A = np.zeros((n, 64))
    # Config B — stats + scale (320 dims)
    X_B = np.zeros((n, 320))
    # Config C — full 582 dims (already built)
    X_C = np.zeros((n, 582))

    for i, (_, row) in enumerate(df.iterrows()):
        if i % 1000 == 0:
            print(f"Ablation: {i:,}/{n:,}...")
```

```

stats = compute_precomputed_stats(row['KPIs'])    # 64
scale = compute_patchwise_scale(row['KPIs'])     # 256
diffs = compute_first_order_differences(row['KPIs']) # 256
cats = compute_categorical_encoding(row['KPIs'])  # 6

X_A[i] = stats
X_B[i] = np.concatenate([stats, scale])
X_C[i] = np.concatenate([stats, scale, diffs, cats])

return X_A, X_B, X_C

```

Step 2.5.2 — Run ablation in notebook

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import f1_score
import numpy as np

results = {}

for name, X in [('Stats-only (64)', X_A),
                ('Stats+Scale (320)', X_B),
                ('Full (582)', X_C)]:

    X_tr, X_te = X[train_idx], X[test_idx]

    model = RandomForestClassifier(
        n_estimators=200,
        class_weight='balanced',
        random_state=42,
        n_jobs=-1
    )
    model.fit(X_tr, y_bin_train)
    y_pred = model.predict(X_te)

    f1 = f1_score(y_bin_test, y_pred)
    results[name] = f1
    print(f'{name}: F1 = {f1:.3f}')

```

```
print("\nScale contribution (Full - Stats only):",
      results['Full (582)'] - results['Stats-only (64)'])
```

PHASE 7 — COMPLETE STREAMLIT DASHBOARD

Goal: Build all 5 panels of the dashboard as specified in the proposal.

Timeline: Week 10–11

Owner: P4 (P1 as backup)

Replace the skeleton in `app/app.py` with this complete version:

```
"""
KPI-RAG Streamlit Dashboard — Complete Version
5 Panels: Detection | SHAP | KPI Signal | Explanation | Sources
"""

import streamlit as st
import numpy as np
import pandas as pd
import plotly.graph_objects as go
import plotly.express as px
import joblib
import shap
import json
import sys
import re
sys.path.append('../src')

from feature_engineering import build_feature_vector, KPI_CHANNELS

# PAGE CONFIG
st.set_page_config(
    page_title="KPI-RAG: 5G Fault Diagnosis",
    page_icon=" ",
```

```

        layout="wide"
    )

#    LOAD RESOURCES
@st.cache_resource
def load_all():
    model1 = joblib.load('./artifacts/models/model1_binary.pkl')
    model2 = joblib.load('./artifacts/models/model2_multiclass.pkl')

    with open('./data/type_to_int.json') as f:
        type_to_int = json.load(f)
    int_to_type = {v: k for k, v in type_to_int.items()}

    with open('./artifacts/alignment_table/alignment_table_draft.json') as f:
        alignment_list = json.load(f)
    alignment_table = {row['fault_type']: row for row in alignment_list}

    import chromadb
    from sentence_transformers import SentenceTransformer
    chroma_client = chromadb.PersistentClient(path="./artifacts/chromadb")
    collection = chroma_client.get_collection("telecomts_tickets")
    encoder = SentenceTransformer('all-MiniLM-L6-v2')

    return model1, model2, int_to_type, alignment_table, collection, encoder

model1, model2, int_to_type, alignment_table, collection, encoder = load_all()

#    HEADER
st.title("    KPI-RAG: 5G Network Fault Diagnosis")
st.markdown("**AI-powered anomaly detection with 3GPP-grounded root cause explanation")
st.divider()

#    FILE UPLOAD
uploaded_file = st.file_uploader(
    "Upload a TelecomTS sample (CSV with KPI columns)",
    type=['csv']
)

if not uploaded_file:
    st.info("Upload a TelecomTS sample CSV file to begin diagnosis.")

```

```

st.stop()

#    PROCESS SAMPLE
df_sample = pd.read_csv(uploaded_file)
st.success(f"File loaded: {len(df_sample)} rows, {len(df_sample.columns)} columns")

# Build feature vector
kpi_text = df_sample.to_dict('records')
feature_vec = build_feature_vector(str(kpi_text))
feature_vec_2d = feature_vec.reshape(1, -1)

# Model 1 prediction
prob_anomaly = model1.predict_proba(feature_vec_2d)[0][1]
is_anomaly = prob_anomaly > 0.5

# Model 2 prediction (only if anomaly)
fault_type = "Normal"
fault_confidence = 0.0
if is_anomaly:
    fault_class = model2.predict(feature_vec_2d)[0]
    fault_confidence = model2.predict_proba(feature_vec_2d)[0].max()
    fault_type = int_to_type.get(fault_class, "Unknown")

#    PANEL 1: DETECTION RESULT
st.subheader(" Panel 1 — Detection Result")
col1, col2, col3 = st.columns(3)

with col1:
    status = " ANOMALY DETECTED" if is_anomaly else " NORMAL"
    st.metric("Status", status)

with col2:
    st.metric("Anomaly Confidence", f"{prob_anomaly*100:.1f}%")

with col3:
    if is_anomaly:
        st.metric("Fault Type", fault_type,
                  delta=f"Confidence: {fault_confidence*100:.1f}%")
    else:
        st.metric("Fault Type", "N/A")

```

```

st.divider()

if not is_anomaly:
    st.success("No anomaly detected in this window.")
    st.stop()

# PANEL 2: KPI SIGNAL PLOT
st.subheader(" Panel 2 — KPI Signal Plot")

kpi_to_plot = st.multiselect(
    "Select KPI channels to display:",
    options=KPI_CHANNELS,
    default=['RSRP', 'DL_BLER', 'UL_SNR']
)

if kpi_to_plot and kpi_to_plot[0] in df_sample.columns:
    fig = go.Figure()
    for kpi in kpi_to_plot:
        if kpi in df_sample.columns:
            fig.add_trace(go.Scatter(
                y=df_sample[kpi].values,
                name=kpi,
                mode='lines'
            ))
    fig.update_layout(
        title="KPI Time Series (128 timesteps)",
        xaxis_title="Timestep",
        yaxis_title="Value",
        height=400
    )
    st.plotly_chart(fig, use_container_width=True)
else:
    st.info("KPI columns not found in uploaded file — showing column names instead")
    st.write(df_sample.columns.tolist())

st.divider()

# PANEL 3: SHAP ATTRIBUTION
st.subheader(" Panel 3 — Feature Attribution (SHAP)")

```

```

explainer = shap.TreeExplainer(model2)
shap_vals = explainer.shap_values(feature_vec_2d)

# Get SHAP for predicted class
predicted_class_idx = model2.predict(feature_vec_2d)[0]
if isinstance(shap_vals, list):
    shap_for_class = shap_vals[predicted_class_idx][0]
else:
    shap_for_class = shap_vals[0]

# Build feature names
feature_names = []
for ch in KPI_CHANNELS:
    for stat in ['mean', 'std', 'min', 'max']:
        feature_names.append(f"{ch}_{stat}")
for ch in KPI_CHANNELS:
    for p in range(8):
        feature_names.extend([f"{ch}_p{p}_mean", f"{ch}_p{p}_std"])
for ch in KPI_CHANNELS:
    for p in range(8):
        feature_names.extend([f"{ch}_diff_p{p}_mean", f"{ch}_diff_p{p}_std"])
feature_names.extend(['UL_TCP', 'UL_UDP', 'UL_None', 'DL_TCP', 'DL_UDP', 'DL_None'])

# Get top 10 features
top_idx = np.argsort(np.abs(shap_for_class))[-10:][::-1]
top_features = [feature_names[i] for i in top_idx]
top_shap = [shap_for_class[i] for i in top_idx]

colors = ['red' if v > 0 else 'blue' for v in top_shap]

fig_shap = go.Figure(go.Bar(
    x=top_shap[::-1],
    y=top_features[::-1],
    orientation='h',
    marker_color=colors[::-1]
))
fig_shap.update_layout(
    title=f"Top 10 KPI Attribution for {fault_type}",
    xaxis_title="SHAP Value (red = increases anomaly probability)",

```



```

        height=400
    )
    st.plotly_chart(fig_shap, use_container_width=True)

    # Extract top 3 KPIs for query
    top3_idx = np.argsort(np.abs(shap_for_class))[-3:][::-1]
    top3_kpis = [feature_names[i].split('_')[0] for i in top3_idx]
    top3_directions = ['increased' if shap_for_class[i] > 0 else 'decreased' for i in top3_idx]
    top3_str = [f"{k} ({d})" for k, d in zip(top3_kpis, top3_directions)]

    st.divider()

    # PANEL 4: RCA EXPLANATION
    st.subheader(" Panel 4 — Root Cause Explanation")

    # Get alignment table row
    alignment_row = alignment_table.get(fault_type)

    if fault_type == 'Jamming':
        st.warning(" Jamming detected — no 3GPP standard applicable. "
                  "This is a physical RF attack. Displaying historical evidence only.")

    # Build and execute RAG query
    query = (f"{fault_type} anomaly detected. "
            f"Primary affected KPIs: {', '.join(top3_str)}. "
            f"Retrieve: root cause, 3GPP standard reference, and resolution.")

    query_embedding = encoder.encode([query]).tolist()
    results = collection.query(
        query_embeddings=query_embedding,
        n_results=3,
        include=['documents', 'metadatas', 'distances']
    )

    retrieved_tickets = results['documents'][0]
    similarities = [1 - d for d in results['distances'][0]]

    # Check low confidence
    if max(similarities) < 0.45:
        st.warning(" Low retrieval confidence — limited historical precedent for this pattern.

```

```

# Generate explanation
with st.spinner("Generating grounded explanation..."):
    from llm_explainer import generate_explanation
    explanation = generate_explanation(
        fault_type=fault_type,
        top_kpis=top3_str,
        retrieved_tickets=retrieved_tickets,
        alignment_row=alignment_row,
        use_groq=True
    )

# Display explanation
col_a, col_b = st.columns(2)
with col_a:
    st.markdown("***Root Cause:**")
    st.info(explanation.get('root_cause', 'N/A'))

    st.markdown("***3GPP Reference:**")
    ref = explanation.get('3gpp_reference', 'N/A')
    if ref and ref != 'null':
        st.success(f" {ref}")
    else:
        st.warning("No 3GPP reference — template fallback used")

with col_b:
    st.markdown("***O-RAN Component:**")
    st.info(explanation.get('oran_component', 'N/A'))

    st.markdown("***Recommended Action:**")
    st.warning(explanation.get('recommended_action', 'N/A'))

    st.markdown("***Reasoning Trace:**")
    st.text(explanation.get('reasoning_trace', 'N/A'))

    if explanation.get('template_fallback'):
        st.caption(" Template-based explanation — LLM generation failed")

st.divider()

```

```
# PANEL 5: SOURCES
st.subheader(" Panel 5 — Retrieved Evidence Sources")

for i, (ticket, sim, meta) in enumerate(zip(
    retrieved_tickets,
    similarities,
    results['metadatas'][0]
)):
    with st.expander(f"Source {i+1} — {meta.get('anomaly_type','Unknown')} "
                    f"(similarity: {sim:.2f})"):
        st.text(ticket[:500] + "..." if len(ticket) > 500 else ticket)

if alignment_row:
    with st.expander(" Alignment Table Row"):
        st.json(alignment_row)
```

PHASE 8.5 — HUGGINGFACE SPACES DEPLOYMENT

Goal: Deploy the dashboard to a public URL.

Timeline: Week 11

Owner: P1

Step 8.5.1 — Create requirements.txt for deployment

```
datasets==2.19.0
pandas==2.2.2
numpy==1.26.4
scikit-learn==1.5.0
shap==0.45.0
chromadb==0.5.0
sentence-transformers==3.0.0
streamlit==1.35.0
plotly==5.22.0
imbalanced-learn==0.12.3
```

```
openai==1.35.0
python-dotenv==1.0.1
```

Step 8.5.2 — Create Dockerfile

Create file `Dockerfile` in project root:

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 7860

CMD ["streamlit", "run", "app/app.py",
    "--server.port=7860",
    "--server.address=0.0.0.0"]
```

Step 8.5.3 — Deploy to HuggingFace Spaces

1. Create account at <https://huggingface.co>
2. New Space Space name: `kpi-rag-5g` SDK: Docker
3. Push your code:

```
git init
git add .
git commit -m "Initial KPI-RAG deployment"
git remote add space https://huggingface.co/spaces/YOUR_USERNAME/kpi-rag-5g
git push space main
```

4. Set your API key as a Space Secret:
 - Settings Variables and Secrets New Secret
 - Name: `GROQ_API_KEY`

- Value: your key
-

PHASE 9 — TELEQNA EVALUATION PROBE

Goal: Use TeleQnA as a secondary probe for RAG explanation quality.

Timeline: Week 12

Owner: P4

Step 9.1 — Load TeleQnA

```
import json
import os

# Download from: https://github.com/netop-team/TeleQnA
# Password: teleqnadataset

with open('data/TeleQnA.json') as f:
    teleqna = json.load(f)

print(f"Total Q&A pairs: {len(teleqna):,}")
print(f"Sample keys: {list(teleqna[list(teleqna.keys())[0]].keys())}")
```

Step 9.2 — Run probe evaluation

```
import chromadb
from sentence_transformers import SentenceTransformer

encoder = SentenceTransformer('all-MiniLM-L6-v2')
chroma_client = chromadb.PersistentClient(path="artifacts/chromadb")
collection = chroma_client.get_collection("telecomts_tickets")

# Sample 50 telecom-relevant questions
```

```
questions = list(teleqna.values())[:50]

correct = 0
for item in questions:
    question = item.get('Question', '')
    correct_answer = item.get('Answer', '')

    # Use RAG to retrieve context
    query_emb = encoder.encode([question]).tolist()
    results = collection.query(query_embeddings=query_emb, n_results=3)
    context = ' '.join(results['documents'][0])

    # Check if correct answer concept appears in retrieved context
    if any(word in context.lower()
            for word in correct_answer.lower().split()[:3]):
        correct += 1

probe_accuracy = correct / len(questions)
print(f"TeleQnA probe accuracy: {probe_accuracy*100:.1f}%")
print("(Proxy for RAG retrieval relevance — not direct standard faithfulness)")
```